

COSC2129/1476
Artificial Intelligence

First-Order Logic
(Predicate Calculus)

Road Map for Today

- ❑ Revision of propositional logic
- ❑ First-order logic (predicate calculus)
 - Models
 - Syntax and semantics
 - Knowledge representation in a simple domain
- ❑ Inference in first-order logic
 - Conversion from first-order to propositional inference
 - Unification and lifted inference rules
 - Forward and backward chaining algorithms

Fundamental Concepts of Logic

- ❑ Syntax and semantics
- ❑ Models and possible worlds
- ❑ Logical entailment ($KB \models \alpha$) and logical inference
- ❑ Model checking
- ❑ Soundness and completeness of an inference algorithm

Propositional Logic: Syntax

Proposition symbols $P, Q, R, \dots$ are atomic sentences

T and F are atomic sentences

If P is a sentence, so is (P) and $[P]$

If P is a sentence, so is $\neg P$

If P and Q are sentences, so are

$P \wedge Q$ (conjunction)

$P \vee Q$ (disjunction)

$P \Rightarrow Q$ (implication) a.k.a. rules or if-then statement

$P \Leftrightarrow Q$ (biconditional)

A **literal** is a atomic sentence either positive (P) or negative ($\neg P$).

Propositional Logic: Semantics

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$
<i>false</i>	<i>false</i>	<i>true</i>	<i>false</i>	<i>false</i>	<i>true</i>	<i>true</i>
<i>false</i>	<i>true</i>	<i>true</i>	<i>false</i>	<i>true</i>	<i>true</i>	<i>false</i>
<i>true</i>	<i>false</i>	<i>false</i>	<i>false</i>	<i>true</i>	<i>false</i>	<i>false</i>
<i>true</i>	<i>true</i>	<i>false</i>	<i>true</i>	<i>true</i>	<i>true</i>	<i>true</i>

Inferences in Propositional Logic

Equivalence

$$\begin{aligned}(\alpha \wedge \beta) &\equiv (\beta \wedge \alpha) && \text{commutativity of } \wedge \\(\alpha \vee \beta) &\equiv (\beta \vee \alpha) && \text{commutativity of } \vee \\((\alpha \wedge \beta) \wedge \gamma) &\equiv (\alpha \wedge (\beta \wedge \gamma)) && \text{associativity of } \wedge \\((\alpha \vee \beta) \vee \gamma) &\equiv (\alpha \vee (\beta \vee \gamma)) && \text{associativity of } \vee \\\neg(\neg\alpha) &\equiv \alpha && \text{double-negation elimination} \\(\alpha \Rightarrow \beta) &\equiv (\neg\beta \Rightarrow \neg\alpha) && \text{contraposition} \\(\alpha \Rightarrow \beta) &\equiv (\neg\alpha \vee \beta) && \text{implication elimination} \\(\alpha \Leftrightarrow \beta) &\equiv ((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)) && \text{biconditional elimination} \\\neg(\alpha \wedge \beta) &\equiv (\neg\alpha \vee \neg\beta) && \text{de Morgan} \\\neg(\alpha \vee \beta) &\equiv (\neg\alpha \wedge \neg\beta) && \text{de Morgan} \\(\alpha \wedge (\beta \vee \gamma)) &\equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma)) && \text{distributivity of } \wedge \text{ over } \vee \\(\alpha \vee (\beta \wedge \gamma)) &\equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma)) && \text{distributivity of } \vee \text{ over } \wedge\end{aligned}$$

Validity

$KB \models \alpha$ if and only if $(KB \Rightarrow \alpha)$ is valid (Deduction Theorem)

Satisfiability

$KB \models \alpha$ if and only if $(KB \wedge \neg\alpha)$ is unsatisfiable (proof by contradiction)

Inferences Rules

Modus Ponens:

$$\frac{\alpha \Rightarrow \beta, \alpha}{\beta}$$

And-Elimination:

$$\frac{\alpha \wedge \beta}{\alpha}$$

Logical equivalence

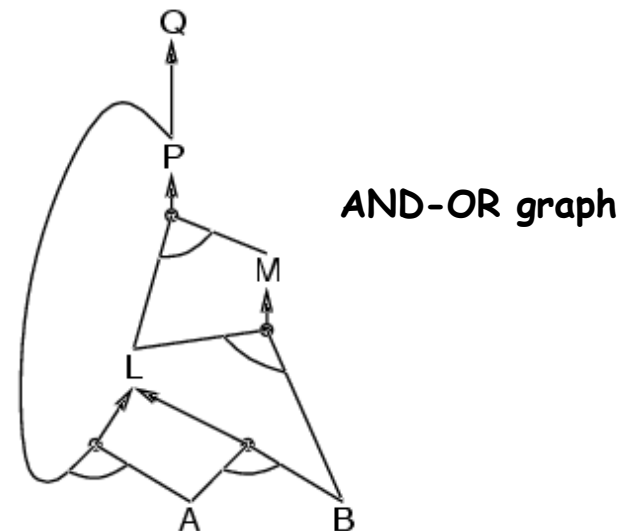
Resolution:

$$\frac{l_i \vee \dots \vee l_k, \quad m_1 \vee \dots \vee m_n}{l_i \vee \dots \vee l_{i-1} \vee l_{i+1} \vee \dots \vee l_k \vee m_1 \vee \dots \vee m_{j-1} \vee m_{j+1} \vee \dots \vee m_n}$$

Inferences Algorithms

- ❑ Model checking
- ❑ Forward chaining and backward chaining

$P \Rightarrow Q$
 $L \wedge M \Rightarrow P$
 $B \wedge L \Rightarrow M$
 $A \wedge P \Rightarrow L$
 $A \wedge B \Rightarrow L$
 A
 B



- ❑ Resolution-based theory proving

Propositional Logic: Pros and Cons

- ☺ Propositional logic illustrates all the basic concepts of logic, and comes with well-developed inference techniques.
- ☺ Propositional logic is **declarative**.
 - Unlike data structures (in programs)
- ☺ Propositional logic allows partial information.
 - Unlike data structures (in programs)
- ☺ Propositional logic is **compositional**:
 - The meaning of $B_{1,1} \wedge P_{1,2}$ is a function of the meaning of $B_{1,1}$ and of $P_{1,2}$
- ☺ Meaning in propositional logic is **context-independent and unambiguous**.
 - Unlike natural language

Propositional Logic: Pros and Cons (cont'd)

☹ Propositional logic (PL) has very limited expressive power.

Consider the following two facts:

1: *The wumpus is dead*; 2: *The wumpus is smelly*.

In PL, these must be represented as different propositional symbols. There is no indication that these two assertions are for the *same* object.

Consider the following two facts:

1: *The wumpus is dead*; 2: *Elvis is dead*.

In PL, these must be represented as different propositional symbols as well. There is no indication that these are the same property of different individuals.

In the wumpus world, we have to represent “*Squares adjacent to pits are breezy*” for each square:

$B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$ Pits cause breezes in adjacent squares.

$B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1})$ Pits cause breezes in adjacent squares.

$B_{1,2} \dots$

$B_{2,2} \dots$

PL lacks the expressive power to describe a complex world in a concise way.

First-Order Logic

- ❑ First-order logic (FOL) is **more expressive** than PL, building on the foundation of PL while borrowing some useful representational ideas (objects, relations, functions) from nature language.
 - **Objects:** people, houses, numbers, colors, baseball games, wars, ...
 - **Relations:** red, round, prime, brother of, bigger than, part of, comes between, ...
 - **Functions:** father of, best friend, one more than, plus, ...
- ❑ Almost any assertion can be regarded as referring to objects and relations.
- ❑ FOL can express facts about **some or all of the objects** in the university so that it can represent **general laws or rules**.
- ❑ The primary difference between PL and FOL is the assumption on the nature of reality (ontological commitment).
 - PL assumes the world consists of facts that do or do not hold.
 - FOL assumes the world consists of objects with certain relations among them that do or do not hold.
- ❑ The possible states of knowledge for each fact (epistemological commitment):
 - True, False and Unknown (PL and FOL)

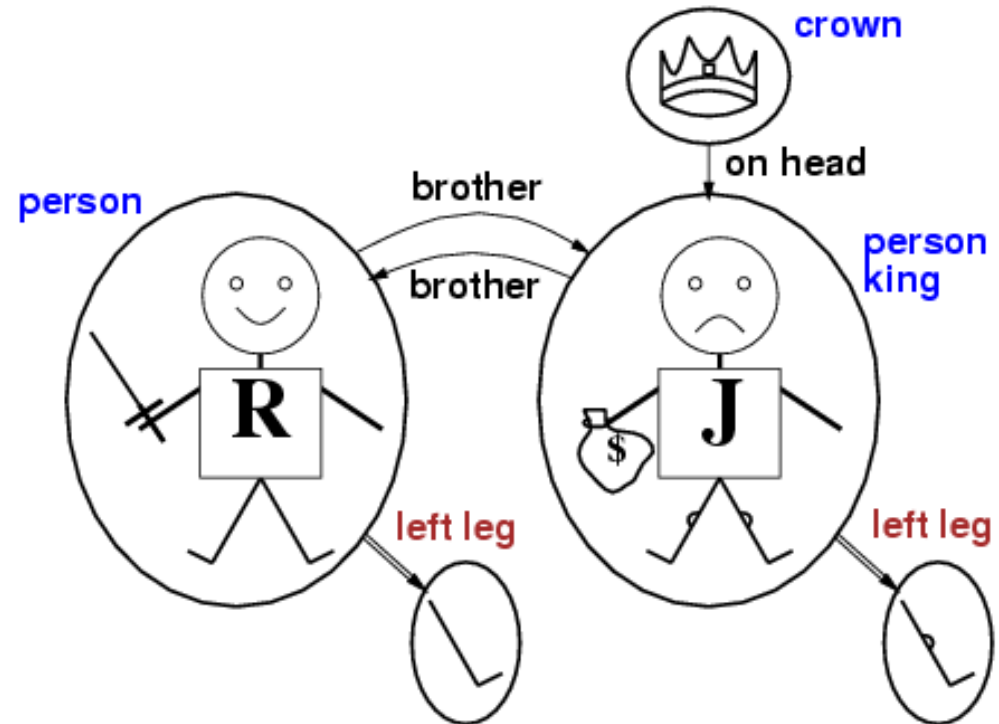
First-Order Logic: Models

- ❑ **Models** are the formal structures that constitutes the possible worlds under consideration, which link the vocabulary of logical sentences to the elements of possible worlds.
- ❑ Basic elements of a **model**:
 - **Objects**: the domain of a model is the set of objects it contains (**non-empty**).
 - **Relations**: the set of tuples of objects that are related.
 - *Tuple*: a collection of objects arranged in a fixed order ($\langle \text{object1}, \text{object2} \rangle$)
 - *Binary relation*: relating pairs of objects
 - *Unary relation (or property)*: property of an object
 - **Functions**: a given object must be related to **exactly one** object.
- ❑ Another essential part of a model is the **link** between these elements and the vocabulary of the logical sentences (syntax).

First-Order Logic: An Example Model

Five objects:

- Richard the Lionheart (King of England from 1189 to 1199)
- His younger brother (the evil King John, who ruled from 1199 to 1215)
- The left legs of Richard and John
- Crown



Relations:

- Binary: “brother”, “on head”
- Unary: “person”, “king”, “crown”

Functions:

- “left leg”

Binary relation examples:

-- “brother”

{<Richard the Lionheart, King John>, <King John, Richard the Lionheart>}

-- “on head”

<the crown, King John>

Function examples:

-- “left leg”: <King John> -> King John’s leg

First-Order Logic: Syntax

Basic **syntactic elements** of FOL:

- Constant symbols: KingJohn, 2, ...
- Predicate symbols: Brother, >, ...
- Function symbols: Sqrt, LeftLegOf, ...

Predicate and function symbols come with **arity** fixing the number of arguments.

A **model** in FOL consists of objects, relations and functions **as well as** an **interpretation** that maps constant symbols to objects, predicate symbols to relations on those objects, and function symbols to functions on those objects.

One possible interpretation with respect to the previous example model (**intended interpretation**):

-- *Richard*: Richard the Lionheart; *John*: the evil King John

-- *Brother*: “brother” relation; *OnHead*: “on head” relation; *Person*, *King*, and *Crown*: the set of objects that are persons, kings and crowns.

-- *Leftleg*: “left leg” function

Model checking is not feasible in FOL!

First-Order Logic: Syntax (cont'd)

Other **syntactic elements** of FOL:

- Variables $x, y, a, b, \dots$
- Connectives $\neg, \Rightarrow, \wedge, \vee, \Leftrightarrow$
- Equality $=$
- Quantifiers $\forall, \exists$

Terms: a logical expression that refers to an object

- ❑ *Constant symbols*
- ❑ *Variables*
- ❑ *Complex term:* a function symbol followed by a parenthesized list of terms
 - the object that is the value of the function applied to arguments
 - Just a complicated kind of name, not “a subroutine call” that “returns a value”
- ❑ *Ground term:* a (complex) term with no variables

First-Order Logic: Sentences

Two kinds of sentences: **atomic sentences** and **complex sentences**

Terms for referring to objects and predicate symbols for referring to relations can be put together to make **atomic sentences (atoms)** that state **facts**.

An **atomic sentence** is formed from a predicate symbol followed by a parenthesized list of terms, e.g., *Brother(Richard, John)*, stating that under the intended interpretation Richard the Lionheart is the brother of King John.

Atomic sentences can have complex arguments, e.g., *Married(Father(Richard), Mother(John))*.

[Semantics]: *An atomic sentence is **true** in a given model if the relation referred to by the predicate symbol holds among the objects referred to by the arguments.*

Complex sentences are made from atomic sentences using connectives, e.g.,

Brother(Richard, John) $\Rightarrow$ Sibling(John, Richard) ; $\neg$ Brother(Father(Richard), John);

Brother(Richard, John) $\wedge$ Brother(John, Richard)

First-Order Logic: Quantifiers

FOL allows objects. To express properties of entire sets of objects, **quantifiers** can be used instead of enumerating the objects by name.

Two standard quantifiers in FOL: **Universal Quantifier** ($\forall$) and **Existential Quantifier** ($\exists$)

General form of **Universal Quantifier**:

$\forall$ <variable> <sentence>

It means the sentence is true for **every** possible value of the variable (i.e., every object), e.g.,

$\forall x \ P(x)$

- for all x, P is true.

$\forall x \ Red(x)$

- all objects are red.

$\forall x \ Likes(x, AI)$

- everyone likes AI.

$\forall x \ Study(x, AI) \Rightarrow Smart(x)$

- everyone studying AI is smart.

[Semantics]: $\forall x \ P$ is true in a given model if P is true in all possible **extended interpretations** constructed from the interpretation given in the model, where each extended interpretation specifies a domain element to which x refers.

First-Order Logic: Quantifiers (cont'd)

General form of **Existential Quantifier**

$\exists$ <variable> <sentence>

It means the sentence is true for **some** possible value of the variable (i.e., some object), e.g.,

$\exists x P(x)$

It means that there exists an x such that $P(x)$ is true or for some x $P(x)$ is true.

$\exists x \text{Red}(x)$

- at least one object is red.

$\exists x \text{Likes}(x, \text{Apple})$

- someone likes apple.

$\exists x \text{Study}(x, \text{Laws}) \wedge \text{Smart}(x)$

- someone studying laws is smart.

[Semantics]: $\exists x P$ is true in a given model if P is true in at least one **extended interpretation** that assigns x to a domain element.

Next, we show some common mistakes when using quantifiers.

Common Mistakes for Quantifiers

Typically, $\Rightarrow$ is the natural connective to use with $\forall$, while $\wedge$ is with $\exists$

Common mistakes:

using $\wedge$ as the connective with $\forall$, using $\Rightarrow$ as the connective with $\exists$

$$\forall x \text{ Study}(x, \text{AI}) \wedge \text{Smart}(x)$$

- It does not mean “everyone studying AI is smart”
- It means “everyone studies AI and everyone is smart”.

$$\exists x \text{ Study}(x, \text{Laws}) \Rightarrow \text{Smart}(x)$$

- This sentence can be true if there is someone who does not study laws
- Because if $\text{Study}(x, \text{Laws})$ is false, the implication is true.

Multiple Quantifiers

Multiple quantifiers are used to express more complex sentences, such as

$\forall x \forall y \text{ Brother}(x, y) \Rightarrow \text{Sibling}(x, y)$ means “Brothers are siblings”

Consecutive quantifiers of the same type can be written as one quantifier with several variables.

$\forall x, y \text{ Brother}(x, y) \Rightarrow \text{Sibling}(x, y)$ is same to $\forall x \forall y \text{ Brother}(x, y) \Rightarrow \text{Sibling}(x, y)$

If there are mixtures, the **order** of quantifiers is very important.

$\forall x \exists y \text{ Loves}(x, y)$

- means “Everybody has someone he/she loves”

$\exists y \forall x \text{ Loves}(x, y)$

- means “There is someone who is loved by everyone”

Parentheses can make it clearer. $\forall x (\exists y \text{ Loves}(x, y))$ and $\exists y (\forall x \text{ Loves}(x, y))$

Connection between $\forall$ and $\exists$

1. $\forall x \neg P \equiv \neg \exists x P$

E.g., $\forall x \neg \text{Likes}(x, \text{Parsnips})$ is same as $\neg \exists x \text{Likes}(x, \text{Parsnips})$
(Everyone dislikes parsnips or no one likes parsnips)

2. $\forall x P \equiv \neg \exists x \neg P$

E.g., $\forall x \text{Likes}(x, \text{IceCream})$ is same as $\neg \exists x \neg \text{Likes}(x, \text{IceCream})$
(Everyone likes ice cream or no one dislikes ice cream)

3. $\neg \forall x P \equiv \exists x \neg P$

E.g., $\neg \forall x \text{Likes}(x, \text{Curry})$ is same as $\exists x \neg \text{Likes}(x, \text{Curry})$
(Not everyone likes curry or someone dislikes curry)

4. $\neg \forall x \neg P \equiv \exists x P$

E.g., $\neg \forall x \neg \text{Likes}(x, \text{Garlic})$ is same as $\exists x \text{Likes}(x, \text{Garlic})$
(Not everyone dislikes garlic or someone likes garlic)

Equality

FOL includes one more way to make atomic sentences, i.e., using **equality symbols** to signify that two terms refer to the same object.

[Semantics]: equality sentence $term_1 = term_2$ is true under an intended interpretation if and only if $term_1$ and $term_2$ refer to the same object.

Equality symbols can be used to state facts about a given function, e.g., $Father(John) = Henry$. It can also be used with negation to insist that two terms are not the same object.

E.g., to say that Richard has at least two brothers:

$$\exists x, y \text{ Brother}(x, \text{Richard}) \wedge \text{Brother}(y, \text{Richard}) \wedge \neg(x = y)$$

Knowledge Representation in FOL

A **domain** in knowledge representation is some part of the world which we wish to express some knowledge.

A set of constants, predicates and functions needs to be determined to fit the domain. (*We assume they have been determined for the early and later examples of predicates/functions*)

E.g., in the Wumpus domain

$$\forall s \text{ Breezy}(s) \Rightarrow \exists r \text{ Adjacent}(r,s) \wedge \text{Pit}(r)$$

Predicates *Breezy*, *Adjacent*, etc. are determined for this domain.

The use of constants, predicates and functions must be consistent within a domain, e.g. don't use *Adjacent* sometime but *NextTo* or *Adjac* next time.

There is currently no general set of predicates and functions that can be used in any domain. Whether we can find such a set remains a major research topic.

Kinship Domain

Express some knowledge in the domain of family relationships, or **kinship**.

Objects in this domain are people; two unary predicates; many binary predicates representing various kinship relations; two functions

$$\forall x \text{ Male}(x) \Leftrightarrow \neg \text{Female}(x)$$

Male and female are disjoint.

$$\forall x, y \text{ Parent}(x, y) \Leftrightarrow \text{Child}(y, x)$$

Parent and child are inverse relations.

$$\forall x, y \text{ Mother}(x, y) \Leftrightarrow \text{Female}(x) \wedge \text{Parent}(x, y)$$

One's mother is one's female parent.

$$\forall x, y \text{ Husband}(x, y) \Leftrightarrow \text{Male}(x) \wedge \text{Spouse}(x, y)$$

One's husband is one's male spouse.

$$\forall x, y \text{ GrandParent}(x, y) \Leftrightarrow \exists z \text{ Parent}(x, z) \wedge \text{Parent}(z, y)$$

A grandparent is a parent of one's parent.

$$\forall x, y \text{ Sibling}(x, y) \Leftrightarrow \neg(x = y) \wedge \exists z \text{ Parent}(z, x) \wedge \text{Parent}(z, y)$$

A sibling is another child of one's parent.

$$\forall x, y \text{ Sibling}(x, y) \Leftrightarrow \neg(x = y) \wedge \exists m, f \neg(m = f) \wedge \text{Parent}(m, x) \wedge \text{Parent}(m, y) \wedge \text{Parent}(f, x) \wedge \text{Parent}(f, y)$$

Definition of full sibling (same father and mother)!!

The above sentences are **axioms** of the kinship domain, and also definitions based on primitive predicates.

Theorems are entailed from axioms, essential to reduce computational costs of deriving sentences.

Kinship Domain (cont'd)

Axioms are not always definitions, sometimes just providing more information about certain predicates, e.g., $\forall x \text{ Person}(x) \Leftrightarrow ?$

FOL allows predicates to be used without completely defining them.

Axioms can be “just plain facts”:

- *Parent(Mary, John)*
- *Parent(George, John)*
- *Spouse(Mary, George)*
- *Male(George)*
- *Female(Mary)*
- *Parent (John, Henry)*
- ...

Then we could answer queries like:

- *Who is the husband of Mary?*
- *Is George Henry's grandparent?*

Fun with FOL Sentences

FOL is quite expressive, and can be fun.

There is a big dog next door

$$\exists x (\textit{Big}(x) \wedge \textit{Dog}(x) \wedge \textit{NextDoor}(x))$$

Jim ate a big cake

$$\exists x (\textit{Big}(x) \wedge \textit{Cake}(x) \wedge \textit{ate}(\textit{Jim}, x))$$

Everybody loves Raymond

$$\forall x (\textit{Person}(x) \Rightarrow \textit{Loves}(x, \textit{Raymond}))$$

The lord of the rings

$$\exists x \forall y \textit{Lord}(x, y) \wedge \textit{Ring}(y)$$

Mission Impossible

$$\neg \exists x, y \textit{Mission}(x) \wedge \textit{Solution}(x, y)$$

Inference in FOL

Can we prove “Socrates is mortal” based on the following knowledge base?

1. $\forall x \text{ Man}(x) \Rightarrow \text{Mortal}(x)$, 2. $\text{Man}(\text{Socrates})$

We can **instantiate** sentence 1, then get

3. $\text{Man}(\text{Socrates}) \Rightarrow \text{Mortal}(\text{Socrates})$

$\text{Man}(x) \Rightarrow \text{Mortal}(x)$ is true for all x , so is true for any instance of x such as *Socrates, John, Richard*

[**Universal Instantiation**]: we can infer any sentence obtained by substituting a ground term for the variable. Here, **Substitution** is $\{x/\text{Socrates}\}$.

We apply Modus Ponens
$$\frac{\alpha \Rightarrow \beta, \alpha}{\beta}$$
 on sentences 3 & 2, then we get

4. $\text{Mortal}(\text{Socrates})$

So, we prove “Socrates is mortal”.

Universal Instantiation

- Every instantiation of a universally quantified sentence is entailed by it:

$$\frac{\forall v \alpha}{\text{SUBST}(\{v/g\}, \alpha)}$$

for any variable v and ground term g

- **Substitution** θ is written as: { <variable> / <instantiation> }.
- $\text{SUBST}(\theta, \alpha)$ denotes the result of applying the substitution θ to the sentence α .
- E.g., $\forall x \text{ King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x)$ yields:
 $\text{King}(\text{John}) \wedge \text{Greedy}(\text{John}) \Rightarrow \text{Evil}(\text{John})$
 $\text{King}(\text{Richard}) \wedge \text{Greedy}(\text{Richard}) \Rightarrow \text{Evil}(\text{Richard})$
 $\text{King}(\text{Father}(\text{John})) \wedge \text{Greedy}(\text{Father}(\text{John})) \Rightarrow \text{Evil}(\text{Father}(\text{John}))$
.
.

Existential Instantiation

- For any sentence α , variable v , and constant symbol k that does not appear elsewhere in the knowledge base:

$$\frac{\exists v \alpha}{\text{Subst}(\{v/k\}, \alpha)}$$

- The variable is replaced a single new constant symbol.
- E.g., $\exists x \text{Crown}(x) \wedge \text{OnHead}(x, \text{John})$ yields:

$$\text{Crown}(C_1) \wedge \text{OnHead}(C_1, \text{John})$$

provided C_1 is a new constant symbol, called a **Skolem constant**

- Existential sentence says that there is some object satisfying a condition, and applying the existential instantiation just gives a name to that object.

Propositionalization

Inference in FOL is an extension of inference in PL.

We can reduce FOL inference to propositional inference via **propositionalization**.

(Need to deal with *substitutions* and *quantifiers*)

The basic idea

- A variable of sentences with existential quantifier can be substituted by one instantiation.
- A variable of sentences with universal quantifier can be substituted by the set of *all possible* instantiations.

Substitution is written as: { <variable> / <instantiation> }.

E.g., $\forall x \text{ Man}(x) \Rightarrow \text{Mortal}(x)$, Substitutions: { $x/\text{Socrates}$ }, { x/John }, { $x/\text{Richard}$ }, ...

$\exists x \text{ Man}(x) \wedge \text{Mortal}(x)$, Substitution: { $x/M1$ }

Generalized Modus Ponens

Modus Ponens can be extended (lifted) for inference in FOL – **Generalized Modus Ponens**.

$$\frac{p'_1, p'_2, \dots, p'_n, (p_1 \wedge p_2 \wedge \dots \wedge p_n) \Rightarrow q}{SUBST(\theta, q)}$$

where θ is a substitution such that for all i , $SUBST(\theta, p'_i) = SUBST(\theta, p_i)$

There are $n+1$ premises to this rule: the n atomic sentences p'_i and the one implication.

It is basically modus ponens + to find a substitution for the variables in sentences.

Generalized Modus Ponens (cont'd)

Example of Generalized Modus Ponens (GMP) - to prove “Socrates is mortal”

1. $\forall x \text{ Man}(x) \Rightarrow \text{Mortal}(x)$ 2. $\text{Man}(\text{Socrates})$

p'_1 is $\text{Man}(\text{Socrates})$, p_1 is $\text{Man}(x)$, q is $\text{Mortal}(x)$

We can find a substitution θ of x to make:

$$\begin{array}{c} \text{SUBST}(\theta, p'_1) = \text{SUBST}(\theta, p_1) \\ \downarrow \\ \text{SUBST}(\theta, \text{Man}(\text{Socrates})) = \text{SUBST}(\theta, \text{Man}(x)) \end{array}$$

The right substitution θ is $\{x/\text{Socrates}\}$

so, $\text{SUBST}(\theta, q) = \text{SUBST}(\theta, \text{Mortal}(x)) = \text{Mortal}(\text{Socrates})$

Now we can apply the modus ponens rule:

$$\frac{\text{Man}(\text{Socrates}) \quad \text{Man}(x) \Rightarrow \text{Mortal}(x)}{\text{Mortal}(\text{Socrates})}$$

Therefore, “Socrates is mortal” is proved.

Unification

Finding substitutions is a key component in FOL inference. The process is called **unification**.

A unification algorithm takes two sentences and returns a unifier (substitution) for them if one exists.

$$UNIFY(p,q) = \theta \quad \text{where } SUBST(\theta, p) = SUBST(\theta, q)$$

Examples:

	Sentence 1	Sentence 2	θ
1	$p(\text{John})$	$p(x)$	$\{x/\text{John}\}$
2	$p(\text{John})$	$p(\text{Richard})$	Failure
3	$p(\text{John}, x)$	$p(y, \text{Mary})$	$\{y/\text{John}, x/\text{Mary}\}$
4	$p(x, x)$	$p(\text{John}, y)$	$\{x/\text{John}, y/\text{John}\}$
5	$p(x, x)$	$p(\text{John}, \text{Richard})$	Failure
6	$p(x)$	$p(y)$	$\{x/y\}$

Unification (cont'd)

As the result of a unification:

- Some variables are replaced with constant symbols (e.g., 1, 3 of previous examples)
- Some (two or more) variables are made identical (e.g., 4, 6 of previous examples)

When a variable would need to be bound to two distinct values, unification returns failure (e.g., example 5)

Once a variable is given a value via unification, that value cannot be replaced.

There could be more than one substitution being returned. E.g., θ for $p(x)$ and $p(y)$ is $\{x/y\}$. However $\{x/John, y/John\}$, $\{x/Mary, y/Mary\}$... can also make $p(x) = p(y)$.

Unification should only return the most general one, which makes the least commitment to constants. It is called **most general unifier** (MGU).

The algorithm for computing MGUs can be found in the textbook.

Example of Inference in FOL

American law says that it is a crime for Americans to sell weapons to hostile nations. The country Nono, an enemy of America, has some missiles, and all of its missiles were sold to it by Colonel West, who is an American. Prove that Colonel West is a criminal.

The approach is:

1. decide predicates and functions
2. construct a knowledge base
3. search for a proof

By forward-chaining:

-- starts with known facts and infer new ones until reaching the goal

Or by backward-chaining:

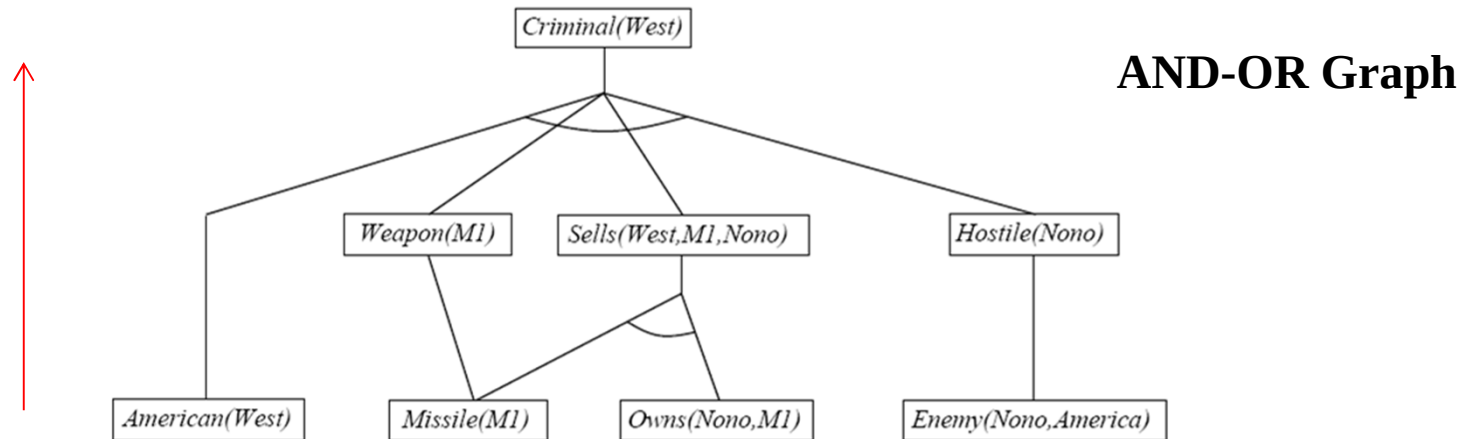
-- starts with the goal, and work backwards until hitting known facts

Example of Inference in FOL (cont'd)

Based on the story and common sense, a knowledge base can be set as:

1. $\forall x, y, z \text{ American}(x) \wedge \text{Weapon}(y) \wedge \text{Sells}(x, y, z) \wedge \text{Hostile}(z) \Rightarrow \text{Criminal}(x)$
... it is a crime for an American to sell weapons to hostile nations:
2. $\text{Owns}(\text{Nono}, M1)$
3. $\text{Missile}(M1)$
Nono ... has some missiles,
4. $\forall x \text{ Missile}(x) \wedge \text{Owns}(\text{Nono}, x) \Rightarrow \text{Sells}(\text{West}, x, \text{Nono})$
... all of its missiles were sold to it by Colonel West
5. $\forall x \text{ Missile}(x) \Rightarrow \text{Weapon}(x)$
Missiles are weapons
6. $\forall x \text{ Enemy}(x, \text{America}) \Rightarrow \text{Hostile}(x)$
An enemy of America counts as "hostile"
7. $\text{American}(\text{West})$
West, who is American ...
8. $\text{Enemy}(\text{Nono}, \text{America})$
The country Nono, an enemy of America ...

Proof by Forward Chaining



-- Start from known facts:

2. *Owns(Nono, M1)* 3. *Missile(M1)* 7. *American(West)* 8. *Enemy(Nono, America)*

-- Infer new facts by applying sentences 4, 5, 6:

Missile(M1) ∧ Owns(Nono, M1) ⇒ Sells(West, M1, Nono) 9

Missile(M1) ⇒ Weapon(M1) 10

Enemy(Nono, America) ⇒ Hostile(Nono) 11

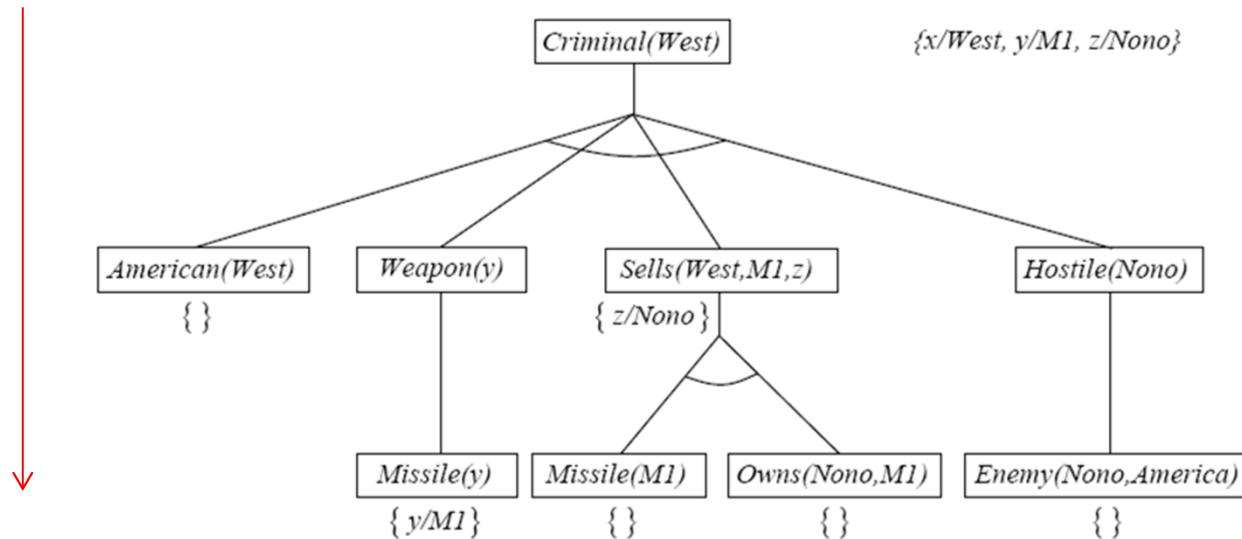
-- Infer the conclusion by applying 1, 7, 9, 10, 11

American(West) ∧ Weapon(M1) ∧ Sells(West, M1, Nono) ∧ Hostile(Nono) ⇒ Criminal(West)

Proof by Backward Chaining

Backward chaining starts from the goal - *Criminal(West)*

The backward chaining process can be represented as a proof tree. The tree should be constructed **depth-first, left to right** (recursively).



The arc across branches mean that each branch must be proved.

A proof branch terminates at an atomic sentence which has no variables.

Proof by Backward Chaining (cont'd)

The goal is *Criminal(West)*.

This can be established by **rule 1** under substitution $\{x/West\}$

American(West) $\wedge$ Weapon(y) $\wedge$ Sells(West,y,z) $\wedge$ Hostile(z) $\Rightarrow$ Criminal(West)

Now each of the following (one branch in the proof tree) must be established:

- *American(West)*
- *Weapon(y)*
- *Sells(West, y, z)*
- *Hostile(z)*

Some of these are in the knowledge base, e.g., *American(West)*, and others requires further backward chaining, e.g., *Weapon(y)*.

To establish *Weapon(y)*, we need establish *Missile(y)* (**rule 5**)

Proof by Backward Chaining (cont'd)

Missile(y) can be established under substitution $\{y/M1\}$ - already in the knowledge base

Next we will establish *Sells(West, M1, z)*. Note y in *Sells(West, y, z)* is already $M1$.

Sells(West, M1, z) can be established under substitution $\{z/Nono\}$ by *Missile(M1)* and *Owns(Nono, M1)* -- already in the knowledge base.

Now we need to establish *Hostile(Nono)*.

It is true because *Enemy(x, America) $\Rightarrow$ Hostile(x)* and *Enemy(Nono, America)* (GMP)

Each predicate required to prove *Criminal(West)* are proved.

So Colonel West is a criminal.

Drawbacks of Logic

Logical consequence (for First-Order Logic) is only semi-decidable.

-- Algorithms exist that say yes to every entailed sentence, but no algorithm exists that also says no to every non-entailed sentence.

Logical proofs can be computationally prohibitive.

-- Problems in Propositional Logic are NP-complete.

Logic does not deal well with uncertainty and default rules.

Tomorrow maybe a sunny day.

Birds fly.

Logic does not provide enough “structure” to the knowledge base creation process.

Conclusion

- ❑ Revision of propositional logic
- ❑ First-order logic
 - Models
 - Syntax and semantics
 - Knowledge representation in a simple domain
- ❑ Inference in first-order logic
 - Conversion from first-order to propositional inference
 - Unification and lifted inference rules
 - Forward and backward chaining algorithms